Programming Problems

Vijay's Mysterious Laundry Counter

At the University of Utah's **Kahlert School of Computing**, Professor **Vijay** has "modernized" the **Hydrothermal Fabrication Laboratory** (the very serious grad-student laundromat).
He installs a big digital display on the wall that shows a single number:

> the **current number of finished laundry loads** for the day.

Several student assistants help run the lab. During the day, they:

- Update the number on the display, and
- Check the number to fill out various forms and reports.

To "avoid crowding around the display," Vijay gives each assistant a quirky set of rules for how they record and apply changes.

How the Assistants Work

There is:

- One **shared display** showing a single integer X (starting from some initial value).
- T assistants, numbered 1 through T.

Each assistant keeps a **personal clipboard** where they write down planned updates **before** walking over to the display.
Each assistant's program is a list of instructions of three kinds:

1. `WRITE v`
   - The assistant writes down a note on their clipboard saying "the display should show `v`".
   - This does **not** change the shared display yet.
   - The assistant keeps all such notes in order on their clipboard.
2. `POST`
   - The assistant walks over to the display and applies **all of their notes**, in order.
   - For each note with value v, they set the display to v.
   - After applying all notes, they clear their clipboard (no notes left).
3. `CHECK`
   - The assistant wants to read what the "current" number of finished loads is.
   - However, they trust their own notes more than the distant display:
     - If their clipboard has **at least one note**, they read the value from the **most recent** note on their clipboard (the last `WRITE` they did that hasn't been cleared by a `POST`).
     - If their clipboard is **empty**, they read the value from the shared display X.
   - Whenever a `CHECK` happens, we want to **record** the number they saw.

Different assistants may have different lists of instructions.

The Daily Schedule

You are also given a **schedule** for the day:

- The schedule is a sequence of assistant IDs.
- At step k of the day, the assistant with ID s_k performs **their next instruction**, if they still have one.
- If an assistant is scheduled but already finished all their instructions, they simply do nothing at that step.

All assistants share the same display X, but each one has their own clipboard of notes.
Your job is to simulate this entire process and find out exactly what numbers were seen during all the `CHECK` operations.

Input Format

1. First line: three integers

   `T P X0`

   - T — number of assistants ($1 \le T \le 10$)
   - P — length of the schedule ($1 \le P \le 100000$)
   - X_0 — initial value shown on the display ($0 \le X_0 \le 10^9$)

2. For each assistant i from 1 to T:

   - One line: integer n_i — the number of instructions for assistant i ($0 \le n_i \le 100000$, and the sum of all n_i over all assistants ≤ 100000)
   - Then n_i lines, each containing one instruction, in one of these forms:
     - `WRITE v` — where $0 \le v \le 10^9$
     - `POST`
     - `CHECK`

3. Final line: P integers

   `s_1 s_2 ... s_P`

   Each s_k is in the range $1..T$ and indicates that at step k, assistant s_k takes their next instruction (if any).

If an assistant has no instructions left when chosen in the schedule, they do nothing for that step.
You may assume that no invalid situations (like negative values) occur.

Output Format

For **every** `CHECK` instruction executed during the schedule (across all assistants), print the value observed on its own line, in the exact order the `CHECK` operations happen.
If there are K `CHECK` operations executed in total, your output should have exactly K lines.

Example

Input

```
2 7 0
3
WRITE 5
CHECK
POST
3
WRITE 10
CHECK
POST
1 2 1 2 1 2 2
```

Explanation
There are 2 assistants. Initial display value: $X = 0$.
Assistant 1's instructions:

1. `WRITE 5`
2. `CHECK`
3. `POST`

Assistant 2's instructions:

1. `WRITE 10`
2. `CHECK`
3. `POST`

Schedule: 1 2 1 2 1 2 2.

- Step 1: assistant 1 → `WRITE 5`
  - Clipboard1: [5], Clipboard2: []
  - Display $X = 0$
- Step 2: assistant 2 → `WRITE 10`
  - Clipboard1: [5], Clipboard2: [10]
  - Display $X = 0$
- Step 3: assistant 1 → `CHECK`
  - Clipboard1 not empty ⇒ sees last note: 5
  - Output: 5
- Step 4: assistant 2 → `CHECK`
  - Clipboard2 not empty ⇒ sees last note: 10
  - Output: 10
- Step 5: assistant 1 → `POST`
  - Applies notes in order: sets $X = 5$
  - Clipboard1 becomes empty

- Step 6: assistant 2 → `POST`
  - Applies notes in order: sets $X = 10$
  - Clipboard2 becomes empty
- Step 7: assistant 2 → no instructions left, does nothing.

Final `CHECK` results, in order:

```
5
10
```

Professor Vijay happily announces that this proves his "laundry coordination rules" are perfectly reasonable.
The students are… not entirely convinced.

John's LaundryScript Verifier

At the University of Utah's **Kahlert School of Computing**, Professor **John** insists that all serious research must now be done in his new domain-specific language:

> **LaundryScript** – "A rigorous language for specifying laundry transformations."

LaundryScript lets you describe how many loads of laundry are in the **dirty basket** over time. Professor John has also written a **LaundryScript-to-WasherCode compiler** that "optimizes" these programs so they run faster on the washing machine.
The only nobody actually trusts the optimizer.
To convince his students the compiler is correct, John wants a tool that checks whether the **original LaundryScript program** and the **optimized version** do the *same thing* to the dirty laundry count.
You've been volunteered.

LaundryScript Semantics

Both programs start with the **same initial number of dirty loads** L (an integer).
Then each program executes a sequence of commands, one per line. The allowed commands are:

- `ADD x`
  Increase the dirty-load count by x. (Think: more loads get dumped in the hamper.)
- `REMOVE x`
  Decrease the dirty-load count by x. (Think: that many loads get fully washed and put away.)
- `DOUBLE`
  Double the current dirty-load count. (Someone dumped a whole extra identical pile onto the existing one.)
- `HALF`
  Replace the current dirty-load count with $\lfloor L/2 \rfloor$ (integer division). (Half the laundry vanished due to "random roommate activity".)

At no point will the dirty-load count go below zero in any test case.

What You Need to Do

You are given:

1. The **initial** dirty-load count L.
2. The **original** LaundryScript program (a sequence of commands).
3. The **optimized** LaundryScript program (another sequence of commands).

Your job is to:

1. Simulate the original program starting from L and compute the final number of dirty loads.
2. Simulate the optimized program starting from the **same** L and compute its final number of dirty loads.

3. Check whether these two final values are **the same**.

If the final dirty-load counts match, you print:

```
VERIFIED
```

The Fabric Purification Sciences Complex at the U

At the University of Utah, the new Fabric Purification Sciences Complex (the U's ultra-serious name for a fancy campus laundromat) is running an experiment in the basement of the MEB. Every student who swipes their UCard to start a wash cycle generates a data point in a log.
For one day, the system records a sequence of integers. Each integer represents the "grime index" of a load of laundry, in the exact order the machines processed them.
Engineers at the lab have noticed that the system behaves most predictably during stretches where this grime index never goes down from one load to the next. In other words, starting from some load and moving forward:

> As long as each new load is at least as dirty as the previous one, the lab considers that part of the log a stable interval.

The number of loads in such an interval is called its *stability score*.
You can start such an interval at any position in the log.
When you hit a load that is less dirty than the previous one, that interval ends.
Your job (before you head to Rice–Eccles for the game) is to help the researchers figure out how "stable" the lab was that day.
They want to know the highest stability score over the entire log.

High-Throughput Textile Decontamination at the Center for High Performance Cleansing (CHPC)

The University of Utah takes pride in innovation, efficiency, and supporting students in every aspect of campus life.
To that end, the university has established:

The Center for High Performance Cleansing (CHPC)

Officially, CHPC is "a multidisciplinary facility for advanced hydrothermal textile decontamination and surfactant dynamics research."
To maximize both sustainability and student success, CHPC runs a single, high-efficiency Decontamination Chamber (washer-dryer unit), and the university would like to use it as effectively as possible.
Throughout the day, students, TAs, and faculty submit laundry "experiments" (loads). Each load has:

- `s`: the earliest time the load can start.
  This is when its owner and their laundry arrive at CHPC.
- `f`: the time the load would finish if started at `s` and runs without interruption.

Once a load starts, it runs continuously until it finishes. No mid-cycle interruptions, no pausing "just for one more sock."
Two loads conflict if their time intervals overlap in any way. Since CHPC is committed to safe and reliable operation, the Decontamination Chamber can only handle one load at a time.
The university wants to show that CHPC significantly increases "experimental throughput" (and also clean clothes) by running as many loads as possible in a day.

Task. What is the maximum number of non-conflicting loads that can be scheduled in the Decontamination Chamber?

Optimal Perks at the Hydrothermal Fabrication Laboratory

The graduate students in the Kahlert School of Computing at the University of Utah basically live in the lab—and in the Hydrothermal Fabrication Laboratory (the department's very serious name for the grad student laundromat).
To keep morale high (and clothing less hazardous), the department offers special laundry perks: things like hypoallergenic detergent, extra-softener cycles, express drying, anti-static treatments, and so on. Each type of perk:

- has a "freshness rating" that measures how much the grad students love/benefit from it, and
- costs a certain number of dollars from the department's laundry subsidy budget.

Every year, the department runs a survey to estimate:

- how much the students value each perk (its freshness rating), and
- how much each perk would cost to offer for the semester.

Unfortunately, the budget is limited.

Given a total budget of N dollars and the freshness ratings and costs for all the laundry perks, determine which combination of perks the department should offer to maximize the total freshness for the grad students.
You may choose at most one of each perk (you cannot choose fractional perks, and you can't "half-offer" a perk).

Scheduling the U's Prime-Time Fabric Purification Slots

At the University of Utah's **Fabric Purification Sciences Complex** (the very serious grad-student laundromat), late-night laundry is in high demand. The lab has a limited number of **prime-time laundry slots** each evening (for example, between 8 pm and midnight), and each slot can only handle a certain number of loads.
To avoid chaos, the lab runs a sign-up system:

- There are `S` graduate students who want to do laundry.
- The evening is divided into `T` **discrete time slots** (e.g., 8:00–8:30, 8:30–9:00, …).
- Each time slot j can handle at most `cap[j]` loads (because there are only so many machines and cycles).
- Each student i:
  - can do **at most one** load of laundry that night, and
  - has a set of time slots they are willing/able to use (based on research meetings, TAing, seminars, etc.).

The lab wants to **maximize the number of students** who actually get a laundry slot, while respecting all the per-slot limits.
Your task is to determine how many students can be successfully scheduled for laundry that night.

Formal Description

You are given:

- An integer `S` — the number of students.
- An integer `T` — the number of time slots.
- A list of `T` integers `cap[1..T]`, where `cap[j]` is the maximum number of laundry loads that can be handled in time slot j.
- For each student i ($1 \leq i \leq S$), you are given:
  - an integer `k_i` — the number of time slots student i is willing/able to use, and
  - a list of `k_i` distinct integers in the range $[1, T]$ indicating which time slots they can attend.

Constraints:

- Each student can be assigned to **at most one** of the time slots they listed.
- Each time slot j can be assigned at most `cap[j]` students.
- If a student is not assigned to any of their listed slots, they don't get to do laundry that night.